



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet 2

JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P00 : Java avancé



Chapitre

2

Les collections en Java

2- Collections en Java

2.1 Qu'est ce qu'une collection ?



- **Une collection est :**
 - Un objet conçu afin de représenter un groupe d'objets.
- **Elle est utilisée dans plusieurs contextes, notamment pour :**
 - Stocker, accéder et manipuler des données.
 - Transmettre des données entre différentes méthodes.
- **Les collections permettent de regrouper des unités de données qui forment un ensemble logique ou naturel. Voici quelques exemples concrets d'utilisation :**
 - Une main de poker, qui est une collection de cartes.
 - Un dossier d'emails, représentant une collection de messages.
 - Un répertoire téléphonique, constitué d'une collection de correspondances entre noms et numéros.

2- Collections en Java

2.2 Généricité (1/12)



- Parfois, on souhaite créer une classe mais on ne souhaite pas préciser le type exact de tel ou tel attribut.
- La généricité est un concept permettant de définir des algorithmes identiques (génériques) qui peuvent être utilisés sur de multiples types de données .
 - Cela permet de réduire la quantité de codes à produire
- En Java, il est possible de créer des méthodes et des classes dont certains types sont des paramètres qui seront résolus au moment de l'invocation et de l'instanciation.
 - L'introduction de la généricité se fait via l'opérateur `< >`
 - On peut définir des **méthodes génériques** sur des classes normales.
 - On peut définir des **classes génériques** avec des méthodes génériques.



2- Collections en Java

2.2 Généricité (2/12)



- Pour utiliser une classe générique, il faut fixer le(s) type(s) générique(s). Cela signifie qu'il faut remplacer les paramètres de type formels (comme T, E, etc.) par des types réels lors de l'instanciation.

- Deux syntaxes sont possibles pour instancier une classe générique :

- La syntaxe classique :

```
MaClasse<String> l1 = new MaClasse<String>();
```

- Depuis Java 7, et grâce à **l'inférence de type** (appelée "opérateur diamant"), il est possible d'écrire :

```
MaClasse<String> l2 = new MaClasse<>();
```



2- Collections en Java

2.2 Généricité (3/12)



Exemple

```
class Box<E> {
    private E item;
    public void add(E item) {
        this.item = item;
    }
    public E get() {
        return item;
    }
}
```

On dit que **Box** est une
classe générique ou
classe paramétrée.

```
class Tool {
    private int size;
    private String nom;
    public Tool(int size, String nom) {
        this.size = size;
        this.nom = nom;
    }
    public String toString() {
        return "outil " + nom + " de "
            + "taille " + size;
    }
}
```

```
public class TestBox {
    public static void main(String[] args) {
        Box<Integer> boite1 = new Box<>();
        boite1.add(42);
        System.out.println(boite1.get());
        Box<Tool> boite2 = new Box<>();
        boite2.add(new Tool(5, "marteau"));
        System.out.println(boite2.get());
    }
}
```

Attention au diamant <> :
Box<Integer> b = new Box();
//instruction non-sûre

Résultat d'affichage

```
42
outil marteau de taille
5
```



2- Collections en Java

2.2 Généricité (4/12)



● Quelques problèmes peuvent se poser :

- Les types paramétrés sont uniquement des types objets (pas de int, char, boolean, etc.).
- A l'intérieur d'une classe/méthode paramétrée par un type E, il est **impossible** d'instancier un objet de type E, de créer un tableau de type E, de dériver une classe générique de Throwable ou ses sous-classes, d'utiliser le type E dans un bloc catch et de déclarer un attribut de classe (static) en utilisant le type E.
- Il n'est pas possible de surcharger une méthode en ne changeant que le type paramétré d'un paramètre

```
public void trier(ArrayList<String> chaines) {  
    // traitement  
}  
public void trier(ArrayList<Integer> nombres) {  
    // traitement  
}
```

Le code ci-dessus ne compile pas !

2- Collections en Java

2.2 Généricité (5/12)



- **Le type générique peut être de n'importe quel type objet (y compris de type paramétré) :**

```
ArrayList<ArrayList<Integer>> tab = new ArrayList<>();
```

- **Méthodes génériques : Utilisées lorsque la portée du type générique ne sort pas en dehors de la méthode**

```
public <E> void fillBoxes(E x, ArrayList<Box<E>> boxes){
    for (Box<E> box : boxes)
        box.add(x);
}
```

- **Puisque Object est la super-classe de toutes les classes, peut-on remplacer la déclaration d'un type générique par Object ?**

```
ArrayList<Object> l = new ArrayList<String>(); // ?
l.add("ali");
l.add(new Object());
```

- **Il n'est pas possible** d'affecter une ArrayList de String à une variable de type ArrayList de Object.
- ArrayList<Object> n'est pas un super-type de ArrayList<E> où E est un type donné.

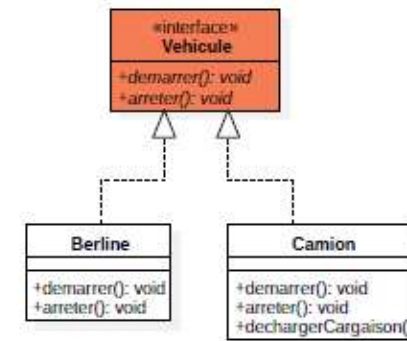
2- Collections en Java

2.2 Généricité (6/12)



```
public class Garage<E extends Vehicule> {
    private ArrayList<E> vehicules;
    public Garage(ArrayList<E> vehicules) {
        this.vehicules = vehicules;
    }
    public void garer(E vehicule) {
        vehicules.add(vehicule);
    }
    public E sortir() {
        return vehicules.remove(0);
    }
}
```

```
public class TestGarage {
    public static void main(String[] args) {
        ArrayList<Camion> v = new ArrayList<>();
        Garage<Camion> g = new Garage<>(v);
        g.garer(new Camion());
    }
}
```



- Dans cet exemple, le mot-clé **extends** veut dire "héritage" au sens général, y compris lorsqu'il s'agit d'une interface.



2- Collections en Java

2.2 Généricité (7/12)



Généricité multiple

```
public class Entry<K,V>{  
    private final K key;  
    private final V value;  
    public Entry(K k, V v){  
        key = k;  
        value = v;  
    }  
    public K getKey(){ return key; }  
    public V getValue(){ return value; }  
    public String toString(){  
        return "(" + key + "," + value + ")";  
    }  
}
```

```
Entry<String, String> grade = new Entry<>("Ali", "B");  
Entry<String, Integer> mark = new Entry<>("Ali", 16);  
System.out.println("grade: " + grade);  
System.out.println("mark: " + mark);
```

```
// Un joker peut être utilisé pour plus de flexibilité :  
Entry<?, ?> r = new Entry<String, Double>("Ali", 0.7);  
Entry<?, ?> r1 = new Entry<>("Ali", "un");
```



2- Collections en Java

2.2 Généricité (8/12)



```
public class ExempleSansJoker {
    public void afficher(ArrayList<Object> liste) {
        for (Object element : liste)
            System.out.println(element);
    }
    public void utiliser() {
        ArrayList<Integer> l = new ArrayList<>();
        afficher(l); // ERREUR de compilation
    }
}
```

```
public class ExempleAvecJoker {
    public void afficher(ArrayList<?> liste) {
        for (Object element : liste)
            System.out.println(element);
    }
    public void utiliser() {
        ArrayList<Integer> l = new ArrayList<>();
        afficher(l); // OK
    }
}
```

● Le **joker** (ou wildcard) $\approx$ type inconnu est noté ?

● Il est utilisé lorsque le code ne dépend pas du type de paramètre.

2- Collections en Java

2.2 Généricité (9/12)



Attention : il n'est pas possible de modifier un objet s'il est paramétré par un joker (verrouillage en insertion).

```
public class TestJoker {
    public void ajouter(ArrayList<Object> liste) {
        liste.add(10); // compile et s'exécute
        liste.add(new Object()); // compile et s'exécute
    }
    public void ajouterJoker(ArrayList<?> liste) {
        liste.add(10); // ERREUR de compilation
        liste.add(new Object()); // ERREUR de compilation
        liste.add(null); // OK
        liste.get(0); // OK
        liste.remove(1); // OK
    }
}
```

Joker restreint (borne supérieure)

- <? extends T>** : le type doit être un sous-type de T ou égal à T.

Joker restreint (borne inférieure)

- <? super T>** : le type doit être un super-type de T ou égal à T.

2- Collections en Java

2.2 Généricité (10/12)



- On définit les classes Automobile, Voiture et VoitureDeCourse qui héritent les unes des autres :

```
public class Automobile { /* ... */ }
public class Voiture extends Automobile { /* ... */ }
public class VoitureDeCourse extends Voiture { /* ... */ }
```

- Il est ainsi possible d'écrire :

```
ArrayList<VoitureDeCourse> v1 = new ArrayList<>();
ArrayList<? extends Voiture> e1 = v1;
```

```
ArrayList<Automobile> v2 = new ArrayList<>();
ArrayList<? super Voiture> e2 = v2;
```

- On crée une instance de ArrayList pour le type Voiture :

```
ArrayList<Voiture> listeVoitures = new ArrayList<>();
```

- On ajoute à cette liste une instance de la classe Voiture et une instance de la classe VoitureDeCourse :

```
listeVoitures.add(new Voiture());
listeVoitures.add(new VoitureDeCourse());
```

2- Collections en Java

2.2 Généricité (11/12)



- Pour accéder à un élément de cette liste, on peut affecter sans risque un élément de cette liste à une variable de type Voiture ou de type Automobile.

```
Voiture voiture = listeVoitures.get(0);
```

```
Automobile auto = listeVoitures.get(1);
```

- Il y a une différence selon que l'on souhaite **accéder** à un élément de cette liste ou **ajouter** un élément à cette liste.

- L'**accès** s'apparente à utiliser le type paramétré comme **paramètre de sortie**, tandis que l'**ajout** s'apparente à utiliser le type paramétré comme **paramètre d'entrée**.

- La liste de Voitures peut être considérée comme :

- une liste de quelque chose qui est **au mieux de type Voiture** lorsque l'on souhaite simplement **accéder** aux éléments de la liste.

- Dans ce scénario, **Voiture** correspond à la **borne supérieure** et on peut écrire l'expression suivante :

2- Collections en Java

2.2 Généricité (12/12)



```
ArrayList<? extends Voiture> lConsult = listeVoitures;
Voiture v = lConsult.get(0);
Automobile a = lConsult.get(1);
```

- une liste de quelque chose qui est **au moins de type Voiture** lorsque l'on souhaite uniquement **ajouter** de nouveaux éléments à la liste.

- Dans ce scénario, **Voiture** correspond à la **borne inférieure** et on peut écrire l'expression suivante :

```
ArrayList <? super Voiture> lAjout = listeVoitures;
lAjout.add(new Voiture());
lAjout.add(new VoitureDeCourse());
```

- Il est envisageable d'utiliser uniquement le joker

```
ArrayList<?> l = listeVoitures;
Voiture v1 = l.get(0); // ERREUR de compilation
l.add(new Voiture()); // ERREUR de compilation
```

- Dans ce scénario, aucune information sur le type paramétré n'est fournie au compilateur.



2- Collections en Java

2.3 Java Collections Framework



- Le Java Collections Framework (**JCF**) définit l'ensemble de la structure des collections en Java, fournissant une architecture unifiée pour représenter et manipuler ces collections.
- Le JCF se compose de plusieurs éléments :
 - Interfaces
 - Types de données abstraits représentant des collections.
 - Permettent de manipuler les collections indépendamment de leur représentation et sont organisées en une hiérarchie unique.
 - Implémentations
 - Implémentations concrètes des interfaces du JCF → Structures de données réutilisables pour accélérer le développement.
 - Algorithmes
 - Méthodes standards pour effectuer des opérations telles que le tri, la recherche, etc.



2- Collections en Java

2.3.1 Interfaces : Structure (1/2)



● Il existe 4 groupes d'interfaces liées aux collections en Java :

1. Collection

- Racine de la structure des collections.
- Représente un groupe d'objets (appelés « éléments »).
- Peut autoriser ou non les doublons d'éléments.
- Peut contenir ou non un ordre intrinsèque.

2. Map

- Un objet qui établit des correspondances entre des clés et des valeurs.
- Ne peut pas contenir de doublons de clés.

3. Iterator

- Permet de gérer les éléments d'une collection de manière séquentielle.



2- Collections en Java

2.3.1 Interfaces : Structure (2/2)



4. Interfaces de comparaison

- Permettent le tri des objets d'un type spécifique implémentant l'interface.
- Il existe deux versions : « **Comparator** » et « **Comparable** »

● Les interfaces du JCF sont organisées dans le package **java.util**. Elles comprennent :

- Deux interfaces de base :
 - **java.util.Collection<E>** et **java.util.Map<K, V>**
- Deux interfaces pour itérer sur les collections :
 - **java.util.Iterator<E>** et **java.util.ListIterator<E>**
- Deux interfaces de comparaison :
 - **java.util.Comparator<E>** et **java.lang.Comparable<E>**



2- Collections en Java

2.3.1.1 Interfaces : Collection (1/7)



- **L'interface Collection<E> est la racine de la structure des collections. Elle fournit un ensemble de méthodes pour manipuler les éléments d'une collection.**
 - **boolean add(E e)** : Ajoute l'élément spécifié à la collection si celui-ci n'est pas déjà présent. Retourne true si la collection est modifiée à la suite de l'ajout, sinon false.
 - **boolean addAll(Collection<? extends E> c)** : Ajoute tous les éléments de la collection spécifiée à cette collection si ceux-ci ne sont pas déjà présents. Retourne true si la collection est modifiée à la suite de l'ajout d'au moins un élément, sinon false.
 - **boolean contains(Object o)** : Retourne true si cette collection contient l'élément spécifié, sinon false.
 - **boolean containsAll(Collection<?> c)** : Retourne true si cette collection contient tous les éléments de la collection spécifiée, sinon elle retourne false.



2- Collections en Java

2.3.1.1 Interfaces : Collection (2/7)



- **void clear()** : Supprime tous les éléments de la collection, la laissant vide.
- **boolean isEmpty()** : Retourne true si cette collection ne contient aucun élément, sinon false.
- **Iterator<E> iterator()** : Retourne un itérateur sur les éléments de cette collection.
- **boolean remove(Object o)** : Supprime la première occurrence de l'élément spécifié de cette collection si celui-ci est présent. Retourne true si la collection est modifiée à la suite de la suppression, sinon false.
- **boolean removeAll(Collection <?> c)** : Supprime tous les éléments de cette collection qui sont également présents dans la collection spécifiée. Retourne true si la collection est modifiée à la suite de la suppression d'au moins un élément, sinon false.
- **Object[] toArray()** : Retourne un tableau contenant tous les éléments de cette collection.

2- Collections en Java

2.3.1.1 Interfaces : Collection (3/7)



- **<T> T[] toArray(T[] a)** : Retourne un tableau contenant tous les éléments de la collection, typé avec le type spécifié. Si le tableau spécifié **a** est assez grand pour contenir les éléments, ils sont stockés dans le tableau et celui-ci est retourné. Sinon, un nouveau tableau de même type est alloué avec la taille de la collection, les éléments sont copiés dans le nouveau tableau, puis le nouveau tableau est retourné.
- **boolean retainAll(Collection<?> c)** : Conserve uniquement les éléments présents dans la collection spécifiée, et supprime les autres éléments de la collection. Retourne true si la collection est modifiée à la suite de la suppression d'au moins d'un élément sinon elle retourne false.
- **int size()** : Retourne le nombre d'éléments contenus dans cette collection.

2- Collections en Java

2.3.1.1 Interfaces : Collection (4/7)



- **default Stream<E> stream()** : Retourne un flux séquentiel de la collection en tant que source de données.
- **default boolean removeIf(Predicate<? super E> filter)** : Supprime tous les éléments de la collection qui correspondent au prédicat spécifié. **Predicate<E>** est une interface fonctionnelle qui fait partie du package `java.util.function`. Elle représente un prédicat (une fonction booléenne) qui accepte un argument et renvoie vrai ou faux en fonction de la condition définie dans le prédicat. En d'autres termes, elle permet de définir une condition ou un critère pour filtrer ou tester des éléments dans une collection ou un flux.

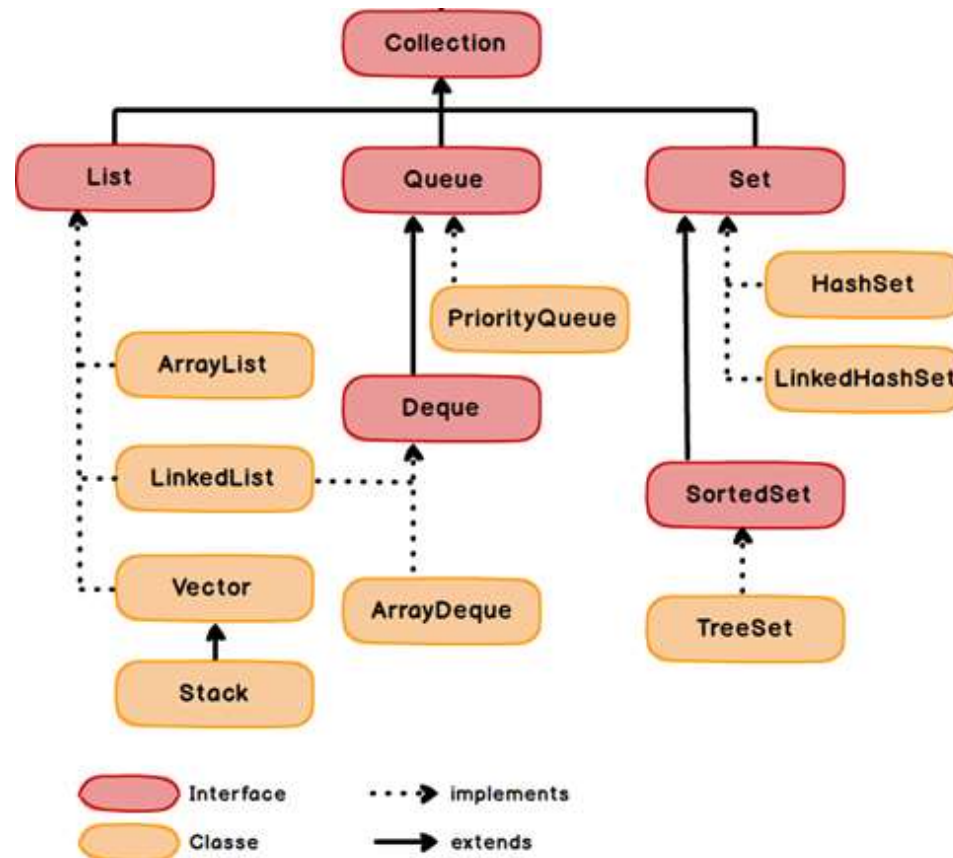
@FunctionalInterface

```
public interface Predicate<T> { boolean test(T t); }
```



2- Collections en Java

2.3.1.1 Interfaces : Collection (5/7)



Extrait du JCF classique



2- Collections en Java

2.3.1.1 Interfaces : Collection (6/7)



● Les principales interfaces dérivées de l'interface `Collection<E>` dans le JCF classique comprennent :

● **`List<E>`** : une structure d'éléments ordonnés

- Permet les doublons.
- Possède un ordre spécifique intrinsèque.
- Offre deux méthodes d'accès particulières :
 - `E get( int index )` : retourne l'élément à l'index spécifié
 - `int indexOf( Object o )` : recherche un objet dans la liste et renvoie son premier index.

● **`Set<E>`** : un ensemble d'éléments

- Interdit les doublons.
- Ne possède pas d'ordre spécifique.
- **`SortedSet<E>`** : une extension de `Set` avec un ordre spécifique



2- Collections en Java

2.3.1.1 Interfaces : Collection (7/7)



- **Queue<E>** : une file d'attente d'éléments
 - Contient l'élément en tête suivi des éléments qui le suivent.
 - Permet les doublons.
 - Possède un ordre spécifique intrinsèque
 - La plupart des implémentations utilisent l'ordre **FIFO** (First In First Out), où les éléments sont ajoutés à la fin de la file et retirés du début.
- **Deque<E>** : une extension de Queue également appelée une double file d'attente :
 - Possibilité de réaliser les opérations d'ajout et de retrait aux deux extrémités de la file.

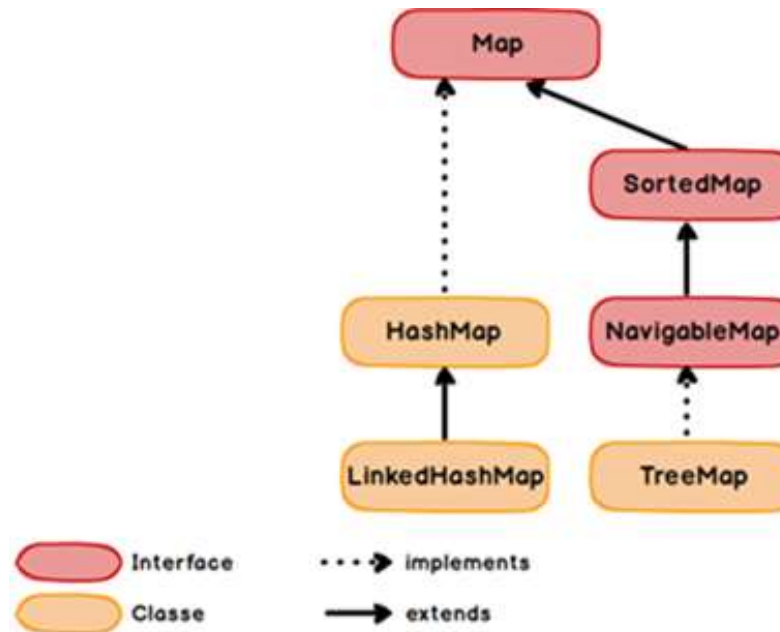


2- Collections en Java

2.3.1.2 Interfaces : Map (1/9)



- Map = une structure de données qui contient des correspondances clé/valeur.
- Dans une Map, les doublons ne sont pas autorisés pour les clés, mais ils peuvent être autorisés pour les valeurs.
- L'interface Map<K, V>, définie dans le JCF classique, fournit les principales méthodes ci-après :



2- Collections en Java

2.3.1.2 Interfaces : Map (2/9)



- **V put(K key, V value)** : Ajoute une association clé-valeur à la map.
- **V get(Object key)** : Retourne la valeur associée à la clé spécifiée.
- **V remove(Object key)** : Supprime l'association clé-valeur correspondante à la clé spécifiée et retourne la valeur associée, ou null si la clé n'existe pas.
- **void clear()** : Supprime toutes les associations clé-valeur de la map.
- **boolean isEmpty()** : Vérifie si la map est vide.
- **boolean containsKey(Object key)** : Vérifie si la map contient la clé spécifiée.
- **boolean containsValue(Object value)** : Vérifie si la map contient la valeur spécifiée.
- **int size()** : Retourne le nombre d'associations clé-valeur dans la map.

2- Collections en Java

2.3.1.2 Interfaces : Map (3/9)



- Une clé peut être associée à plusieurs valeurs dans une map, ce qui est souvent représenté comme : **Map<E1, List<E2>>**.
- Différentes sous-interfaces de Map offrent des fonctionnalités supplémentaires :
 - **SortedMap** : associe des clés triées.
 - **NavigableMap** : permet la navigation dans une SortedMap selon l'ordre ascendant ou descendant des clés.
- L'interface statique **Map.Entry<K, V>** est imbriquée dans l'interface **Map<K, V>** et représente les entrées (associations clé/valeur) d'un objet map.
 - Les objets Map.Entry sont destinés à une utilisation temporaire pendant l'itération à travers les entrées d'une map et ne sont pas conçus pour être stockés ou réutilisés en dehors de cette itération spécifique.



2- Collections en Java

2.3.1.2 Interfaces : Map (4/9)



- Les objets `Map.Entry` offrent la possibilité de modifier la valeur associée à une clé donnée à l'intérieur de cette itération.
- l'interface `Map<K, V>` contient également la méthode **`Set<Map.Entry<K, V>> entrySet()`** qui renvoie un ensemble `Set` contenant toutes les entrées (associations clé-valeur) de la map.
- L'interface `Map.Entry<K, V>` expose les méthodes principales suivantes :
 - **`K getKey()`** : Renvoie la clé de l'entrée de la map.
 - **`V getValue()`** : Renvoie la valeur associée à la clé de l'entrée de la map.
 - **`V setValue(V value)`** : Permet de modifier la valeur associée à la clé de l'entrée de la map. Retourne la valeur précédemment associée à cette clé avant sa modification.

2- Collections en Java

2.3.1.2 Interfaces : Map (5/9)



● Quelques autres méthodes spécifiques aux Maps :

- **default boolean remove(Object key, Object val) :**
Supprime l'association (key, val) de la map et renvoie true si cette association existe, sinon elle renvoie false.
- **default V getOrDefault(Object key, V dval) :**
Renvoie la valeur associée à la clé spécifiée dans la map. Si la clé n'est pas présente dans la map, la méthode renvoie la valeur par défaut dval.
- **Set keySet() :** Renvoie une **vue** Set des clés contenues dans cette map.
- **default V replace(K key, V val) :** Remplace l'association existante avec la clé key par une nouvelle association (key, val) et renvoie l'ancienne valeur. Si aucune association avec la clé key n'existe, aucune action n'est effectuée et null est renvoyé.



2- Collections en Java

2.3.1.2 Interfaces : Map (6/9)



- **Collection<V> values()** : Renvoie une **vue** en tant que Collection contenant toutes les valeurs de la map. L'objet Collection et l'objet map actuel sont liés l'un à l'autre, ce qui signifie que les modifications apportées à la map affecteront la Collection et vice versa.
 - L'ajout ou la suppression d'un élément dans map entraînera l'ajout ou la suppression d'un élément correspondant dans la Collection.
 - La modification de la valeur d'un élément dans la map affectera également la Collection.
 - La suppression d'éléments de la Collection entraînera la suppression des associations correspondantes de la map.
 - Il est important de noter que cet objet Collection **ne prend pas en charge** les opérations **add** et **addAll**.



2- Collections en Java

2.3.1.2 Interfaces : Map (7/9)



● default V putIfAbsent(K key, V value) :

- Si la clé est absente de la map ou associée à la valeur null, l'association clé-valeur est ajoutée à la map et la méthode retourne null.
- Si la clé est déjà présente avec une valeur non nulle, l'association existante est conservée et la méthode retourne la valeur associée à cette clé.

● default V computeIfAbsent(K key, Function<? super K, ? extends V> fct) :

- Si la clé est absente de la map, la fonction fournie est appliquée à la clé afin de calculer une valeur.
 - Si la valeur calculée est non nulle, elle est ajoutée à la map et retournée.
 - Si la valeur calculée est nulle, aucune entrée n'est ajoutée et la méthode retourne null.
- Si la clé est déjà présente, la méthode retourne la valeur existante sans modifier la map.



2- Collections en Java

2.3.1.2 Interfaces : Map (8/9)



● **default V computeIfPresent(K key, BiFunction < ? super K, ? super V, ? extends V> fct) :**

- Si la clé est présente dans la map, la fonction est appliquée à la clé et à sa valeur actuelle :
 - si la valeur calculée est non nulle, la valeur associée à la clé est mise à jour ;
 - si la valeur calculée est nulle, l'entrée correspondante est supprimée de la map.
- Si la clé est absente, la méthode ne modifie pas la map et retourne null.



2- Collections en Java

2.3.1.2 Interfaces : Map (9/9)



● **default V compute(K key, BiFunction<? super K, ? super V,? extends V> fct) :**

- Si la clé est absente, la fonction est appliquée avec la clé et null comme valeur associée :
 - si la fonction retourne une valeur non nulle, l'entrée est ajoutée à la map ;
 - si la fonction retourne null, aucune entrée n'est ajoutée.
- Si la clé est présente, la fonction est appliquée à la clé et à sa valeur actuelle :
 - si la valeur retournée est non nulle, la valeur est mise à jour ;
 - si la valeur retournée est nulle, l'entrée est supprimée de la map..



2- Collections en Java

2.3.1.3 Interfaces : Iterator (1/6)



● L'interface **Iterable** (`java.lang.Iterable<E>`) est une interface racine de l'arborescence des collections.



- Elle étend l'interface `Collection<E>`
- Une classe qui implémente l'interface **Iterable** peut être itérée avec la boucle `foreach`.
- Cette interface fonctionnelle **Iterable<E>** définit :
 - La méthode abstraite **iterator()**, qui renvoie un objet `Iterator<E>` permettant d'itérer sur les éléments de l'objet implémentant l'interface **Iterable**.
 - La méthode par défaut **forEach()** qui permet d'itérer de manière concise sur les éléments de l'itérable. Elle prend en paramètre un objet `Consumer` (généralement exprimée sous forme d'une expression lambda ou d'une référence de méthode). En combinant `forEach()` avec `Consumer`, on peut spécifier une action à effectuer sur chaque élément de l'itérable.



2- Collections en Java

2.3.1.3 Interfaces : Iterator (2/6)



- **Toute collection peut fournir son propre itérateur (objet Iterator) en implémentant la méthode iterator().**
- **L'interface Iterator (java.util.Iterator<E>) permet de parcourir les éléments d'une collection et de supprimer des éléments de la collection pendant une itération. Elle définit les méthodes suivantes :**
 - **boolean hasNext() :** Indique s'il reste des éléments non parcourus dans la collection après l'élément actuel. Elle renvoie true s'il y a au moins un élément suivant, sinon false.
 - **E next() :** Renvoie l'élément suivant de la collection et avance le curseur de l'itérateur vers cet élément suivant. Si aucun élément suivant n'est présent, elle lève une exception NoSuchElementException.
 - **default void remove() :** Supprime l'élément actuel de la collection sous-jacente. Elle peut être appelée **uniquement** après l'appel à la méthode next(), sinon elle lève une exception IllegalStateException.

2- Collections en Java

2.3.1.3 Interfaces : Iterator (3/6)



- **default void forEachRemaining(Consumer<? super E> action) :** Exécute l'action spécifiée sur chaque élément restant dans la collection jusqu'à ce que tous les éléments soient traités ou que l'action lance une exception. Elle est utilisée pour appliquer une action à tous les éléments restants sans nécessiter explicitement une boucle while ou for.

- Consumer est une interface fonctionnelle, du package java.util.function, qui contient la méthode abstraite nommée accept, prenant un argument de type E et ne renvoyant rien. Cette méthode accept est utilisée pour effectuer une action sur l'objet passé en argument.

```
static void filter(Collection<Integer> c) {
    for(Iterator<Integer> it=c.iterator(); it.hasNext();){
        if(it.next() == 3){
            it.remove();
        }
    }
    c.iterator().forEachRemaining(System.out::println);
}
```

2- Collections en Java

2.3.1.3 Interfaces : Iterator (4/6)



● L'interface **ListIterator** est une sous-interface d'**Iterator**.

- Elle est spécialement conçue pour parcourir les éléments d'une **liste**.
- Contrairement à **Iterator**, **ListIterator** prend en charge les déplacements dans les deux sens (avant et arrière) à l'intérieur de la liste (**next()** et **previous()**).
- **ListIterator** permet l'insertion, la suppression ou la mise à jour d'éléments pendant l'itération.

public interface ListIterator<E> extends Iterator<E>

● L'interface **ListIterator** comprend les méthodes suivantes :

- **void add(E e)** : Insère un élément dans la liste juste avant l'élément renvoyé par la méthode **next()**.
- **boolean hasNext()** : Retourne true si la liste a un élément suivant.
- **boolean hasPrevious()** : Retourne true si la liste a un élément précédent.

2- Collections en Java

2.3.1.3 Interfaces : Iterator (5/6)



- **E next()** : Retourne l'élément suivant de la liste. L'exception « NoSuchElementException » est levée si la liste n'a pas d'élément suivant à fournir.
- **int nextIndex()** : Retourne l'index de l'élément qui serait retourné par un appel à next(). Si le curseur est à la fin de la liste, elle retourne la taille de la liste.
- **E previous()** : Retourne l'élément précédent de la liste. L'exception « NoSuchElementException » est levée si la liste n'a pas d'élément précédent.
- **int previousIndex()** : Retourne l'index de l'élément qui serait retourné par un appel à previous(). Si le curseur est au début de la liste, elle retourne -1.
- **void remove()** : Supprime l'élément courant de la liste. L'exception « IllegalStateException » est levée si cette méthode est appelée sans être précédée par un appel à next() ou previous().



2- Collections en Java

2.3.1.3 Interfaces : Iterator (6/6)



- **void set(E e)** : Remplace l'élément renvoyé par la méthode next() ou previous() par l'élément e spécifié. Cette opération est optionnelle et peut être réalisée uniquement si ni remove() ni add(E) n'ont été appelées après le dernier appel à next() ou previous(). L'exception « IllegalStateException » est levée si la méthode set() est appelée avant que next() ou previous() soit invoquée.

Résultat d'affichage

```
import java.util.*;
public class ListIteratorExemple{
    public static void main(String[] args) {
        ArrayList<String> myList = new ArrayList<>();
        myList.add("E1");myList.add("E2");myList.add("E3");
        System.out.println("1. "+myList);
        ListIterator<String> listIt = myList.listIterator();
        listIt.add(new String("E0")); System.out.println("2. "+myList);
        listIt.next(); listIt.set("E5"); System.out.println("3. "+myList);
        listIt.next(); listIt.remove(); System.out.println("4. "+myList);
    }
}
```


2- Collections en Java

2.3.1.4 Interfaces : Comparaison (1/8)



● Deux interfaces fonctionnelles utilisées pour définir des méthodes de comparaison :

1. Comparable :

- Présente dans le package **java.lang**.
- Utilisée pour définir un ordre naturel de tri pour les objets d'une classe.
- Une classe implémente l'interface Comparable pour indiquer qu'elle peut être triée selon un ordre spécifique défini par la méthode `compareTo(Object o)`.
- **public int compareTo(Object o) :** Compare l'objet courant avec un autre objet o spécifié et renvoie un entier :
 - 1 si l'objet courant est considéré comme plus grand que l'objet o fourni en argument.
 - 0 si l'objet courant est considéré comme égal à l'objet o fourni en argument.
 - -1 si l'objet courant est considéré comme plus petit que l'objet o fourni en argument.

2- Collections en Java

2.3.1.4 Interfaces : Comparaison (2/8)



2. Comparator :

- Présente dans le package **java.util**
- Utilisée pour définir des règles de comparaison personnalisées entre des objets qui nécessitent différents critères de tri.
- Les comparateurs sont souvent utilisés lorsque on a besoin de plusieurs façons de trier une classe ou lorsque on ne peut pas modifier la classe des objets à trier.
- La classe qui implémente l'interface Comparator fournit la méthode `compare(T o1, T o2)`
- **public int compare(T o1, T o2) :** Compare deux objets et renvoie un entier :
 - 1 si o1 est considéré comme plus grand que o2.
 - 0 si o1 est considéré comme égal à o2.
 - -1 si o1 est considéré comme plus petit que o2.

2- Collections en Java

2.3.1.4 Interfaces : Comparaison (3/8)



Exemple d'implémentation de Comparable

```
public class Personne implements Comparable {  
    private String  firstName, lastName;  
  
    public Personne(String firstName, String lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
  
    public int compareTo(Object o) {  
        Personne n = (Personne) o;  
        int lastCmp = lastName.compareTo(n.lastName);  
        if(lastCmp==0)  
            lastCmp = firstName.compareTo(n.firstName);  
        return lastCmp;  
    }  
}
```





2- Collections en Java

2.3.1.4 Interfaces : Comparaison (4/8)



● Depuis Java 1.5, l'interface Comparable peut être générique

```
public class Personne implements Comparable<Personne> {  
    private String  firstName, lastName;  
  
    public Personne(String firstName, String lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
  
    public int compareTo(Personne n) {  
        int lastCmp = lastName.compareTo(n.lastName);  
        if(lastCmp==0)  
            lastCmp = firstName.compareTo(n.firstName);  
        return lastCmp;  
    }  
}
```



2- Collections en Java

2.3.1.4 Interfaces : Comparaison (5/8)



● Certaines classes supposent que leurs éléments peuvent être **comparés selon un ordre naturel spécifique**, qui est déterminé par l'implémentation de l'interface **Comparable**. Par exemple :

- Les objets Byte, Short, Integer, Long, Float, Double, BigInteger et BigDecimal suivent un ordre numérique signé.
- Les objets Character suivent un ordre numérique non signé, qui correspond à l'ordre alphabétique des caractères dans le standard Unicode.
- Les objets String suivent un ordre lexicographique.
- Les objets Date suivent un ordre chronologique.
- Les objets File suivent un ordre lexicographique basé sur le chemin d'accès.



2- Collections en Java

2.3.1.4 Interfaces : Comparaison (6/8)



- **L'interface `Comparator<T>` définit plusieurs méthodes pour la composition et la manipulation des comparateurs :**
 - **`default Comparator<T> reversed()` :** Retourne un comparateur qui impose l'ordre inverse du comparateur actuel.
 - **`static <T extends Comparable<? super T>> Comparator<T> reverseOrder()` :** Retourne un comparateur qui impose l'ordre inverse sur les objets qui implémentent l'interface `Comparable`.
 - **`static <T,U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)` :** Retourne un comparateur basé sur l'application d'une fonction d'extraction de clé à chaque élément comparé.
 - **`default Comparator<T> thenComparing(Function<? super T, ? extends U> keyExtractor)` :** Retourne un comparateur qui compare les éléments par leur clé extraite à l'aide de la fonction spécifiée après avoir utilisé le comparateur principal.

2- Collections en Java

2.3.1.4 Interfaces : Comparaison (7/8)



- **static <T,U> Comparator<T> comparing(Function<? super T,? extends U> keyExtractor, Comparator<? super U> keyComparator) :** Retourne un comparateur basé sur une clé extraite à l'aide d'une fonction spécifiée, et de spécifier un comparateur supplémentaire à utiliser si deux clés extraites sont égales selon le comparateur actuel.
- **static <T> Comparator<T> comparingXXX(ToXXXFunction<? super T> keyExtractor) :** Ces méthodes statiques prennent un extracteur de clé comme argument. Chacune retourne un comparateur basé sur la clé extraite à partir des objets à comparer. **XXX** représente le type de la clé extraite qui peut être **Int**, **Long** ou **Double**.
 - Par exemple, **comparingInt()** prend un extracteur de type **ToIntFunction** et retourne un comparateur basé sur une valeur entière extraite.



2- Collections en Java

2.3.1.4 Interfaces : Comparaison (8/8)



- **default Comparator<T> thenComparingXXX(ToXXXFunction<? super T> keyExtractor)** : des méthodes de composition qui permettent de créer un comparateur composite. Elles prennent également un extracteur de clé comme argument et permettent de spécifier un critère de comparaison supplémentaire après avoir utilisé le comparateur principal.
- **default Comparator<T> thenComparing(Comparator<? super T> other)** : Retourne un comparateur qui compare les éléments à l'aide du comparateur spécifié si les objets comparés sont égaux selon le comparateur actuel.
- ```
Comparator<Person> comparator =
Comparator.comparingInt(Person::getAge)
.thenComparingInt(Person::getId);
```

  - Dans cet exemple, les personnes sont d'abord comparées par leur âge, et si deux personnes ont le même âge, alors elles sont comparées par leur identifiant.

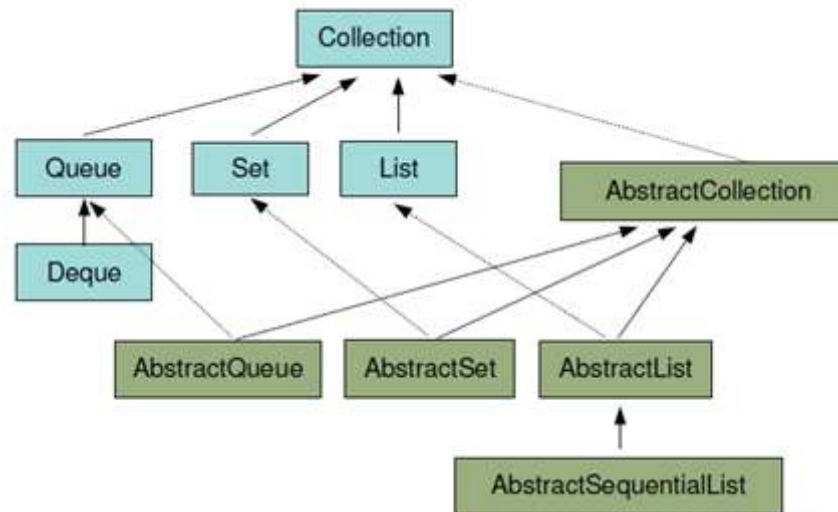




## 2- Collections en Java

### 2.3.2 Implémentations : Structure

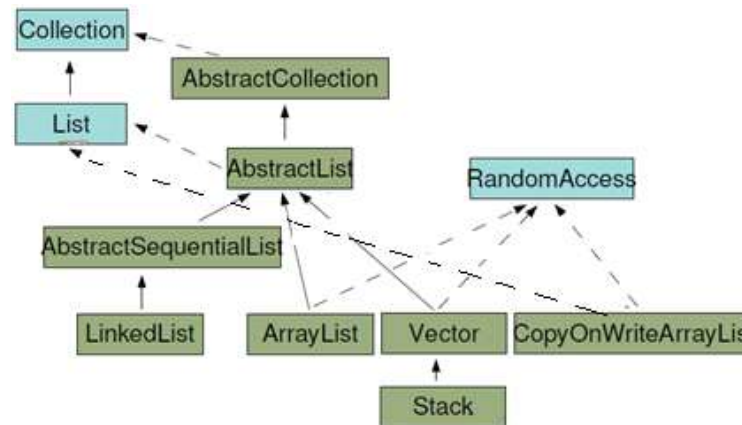
- Chaque collection possède une classe abstraite qui est la classe mère de plusieurs implémentations du même type.



- Les implémentations sont les types d'objets réels utilisés pour stocker des collections.
- Toutes les classes fournies implémentent une ou plusieurs interfaces de base des collections.

## 2- Collections en Java

### 2.3.2.1 Implémentations : List (1/3)



● Deux principales implémentations de l'interface « List » pour gérer les listes indexées.

#### ● ArrayList

- La plus utilisée
- Offre un accès positionnel à vitesse constante
  - ➔ Bien rapide pour les accès positionnels (aléatoire)

#### ● **Vector** : supporte le multithreading

- coûteux à l'utilisation : son utilisation est déconseillée



## 2- Collections en Java

### 2.3.2.1 Implémentations : List (2/3)



#### ● LinkedList

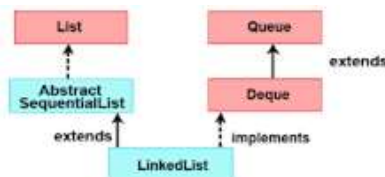
##### ● A utiliser pour

- Parcourir la liste dans les deux sens (liste doublement chaînée).
- Ajouter des éléments au début de la liste.
- Supprimer des éléments au milieu de la liste en cours d'itération.

##### ● Plus lente en termes d'accès positionnel.

##### ● Plus performante en termes d'ajout et de suppression des éléments.

##### ● La classe LinkedList implémente également les interfaces Queue et Deque



```
Queue<String> q = new LinkedList<>();
```



## 2- Collections en Java

### 2.3.2.1 Implémentations : List (3/3)

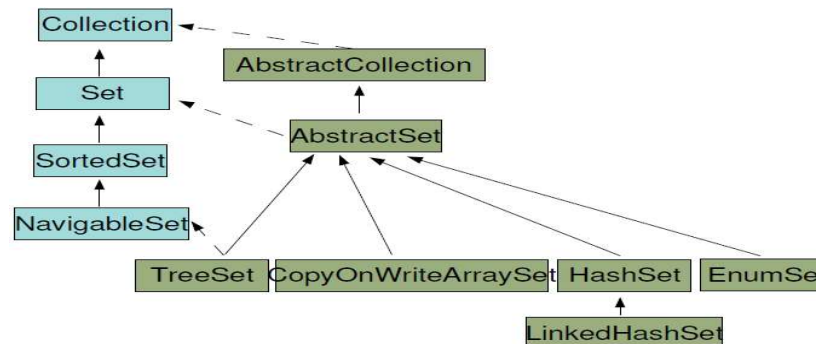


#### ● Opérations spécifiques aux listes :

- Obtenir la position de l'objet spécifié
  - int **indexOf**(Object o) § int **lastIndexOf**(Object o)
- Récupérer l'élément à la position spécifiée
  - E **get**(int i)
- Insérer l'élément spécifié à la position spécifiée
  - void **add**(int i, E elt)
- Remplacer l'élément à la position spécifiée dans la liste par l'élément spécifié.
  - E **set**(int i, E elt)
- Supprimer l'élément à la position spécifiée
  - E **remove**(int i) § void **removeRange**(int froma, int tob)
- Sous-liste : List<E> **subList**(int froma, int tob)

## 2- Collections en Java

### 2.3.2.2 Implémentations : Set (1/3)



#### ● Trois principales implémentations de l'interface « Set » :

##### ● HashSet

- Plus rapide que TreeSet car elle utilise une table de hachage pour stocker les éléments.
- N'offre aucune garantie en termes d'ordre des éléments. Les éléments ne sont pas stockés dans un ordre spécifique.

##### ● LinkedHashSet

- Présente les mêmes caractéristiques et des performances similaires à HashSet mais préserve l'ordre d'insertion des éléments.

## 2- Collections en Java

### 2.3.2.2 Implémentations : Set (2/3)



#### ● TreeSet

- Utilise un arbre pour stocker les éléments.
- Maintient les éléments dans un ordre trié selon l'ordre naturel ou selon un comparateur fourni lors de la création du TreeSet.
- Moins rapide que les autres implémentations car elle doit maintenir un arbre trié.
- Des méthodes spécifiques à l'interface **SortedSet<E>** :
  - `headSet(E e)` : Renvoie une vue de l'ensemble qui contient tous les éléments strictement inférieurs au paramètre spécifié (non inclus).
  - `tailSet(E e)` : Renvoie une vue de l'ensemble qui contient tous les éléments supérieurs ou égaux au paramètre spécifié.
  - `subSet(E fromElement, E toElement)` : Retourne une vue de l'ensemble comprenant tous les éléments compris entre `fromElement` (inclus) et `toElement` (exclus).

## 2- Collections en Java

### 2.3.2.2 Implémentations : Set (3/3)



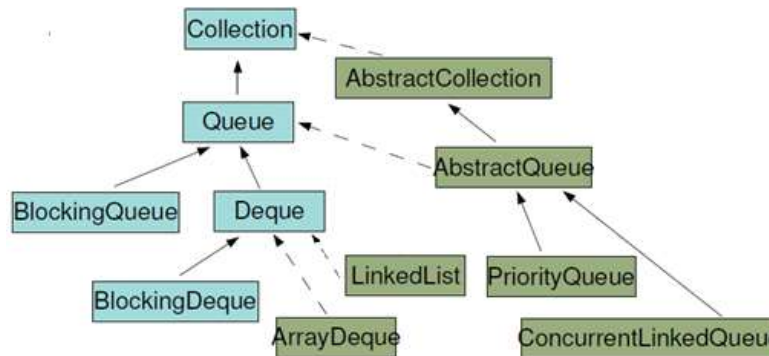
#### ● Les principales méthodes spécifiques aux implémentations de l'interface **Set<E>** sont :

- Unicité des éléments :
  - boolean **add(E e)** : Ajoute l'élément spécifié à l'ensemble si ce dernier ne le contient pas déjà. Renvoie true si l'ensemble a été modifié suite à cette opération, sinon false.
  - L'unicité des éléments est garantie dans un ensemble, ce qui signifie qu'un élément ne peut pas être présent plusieurs fois dans le même ensemble.
- Opérations ensemblistes : **addAll(...)**, **containsAll(...)**, **removeAll(...)**, **retainAll(...)**
- Comparaison d'ensembles : **equals(Object o)** et **hashCode()**
- Conversion en tableau spécifique à l'ensemble : **toArray()**



## 2- Collections en Java

### 2.3.2.3 Implémentations : Queue (1/2)



● Deux principales implémentations de l'interface « Queue » :

#### ● PriorityQueue

- Les éléments sont traités par ordre de priorité. Les éléments sont ordonnés en fonction de leur **ordre naturel** ou en utilisant un **comparateur** fourni lors de la création de la file de priorité.
- L'élément de priorité la plus élevée sera le premier à être récupéré lors de l'appel à poll() ou remove().

#### ● ArrayDeque

- ArrayDeque est une implémentation de Queue et de Deque





## 2- Collections en Java

### 2.3.2.3 Implémentations : Queue (2/2)



- Permet l'insertion et la suppression d'éléments des deux côtés :
  - Insérer des éléments à la fois en tête (push() et offerFirst()) et en queue (offer() et offerLast()) de la deque, et supprimer des éléments à la fois de la tête (pop(), poll() et pollFirst()) et de la queue (pollLast()).
- Implémente une pile LIFO ou une file d'attente FIFO, en fonction de la manière dont les éléments sont manipulés.
- Conçue pour remplacer la classe Stack devenue obsolète.

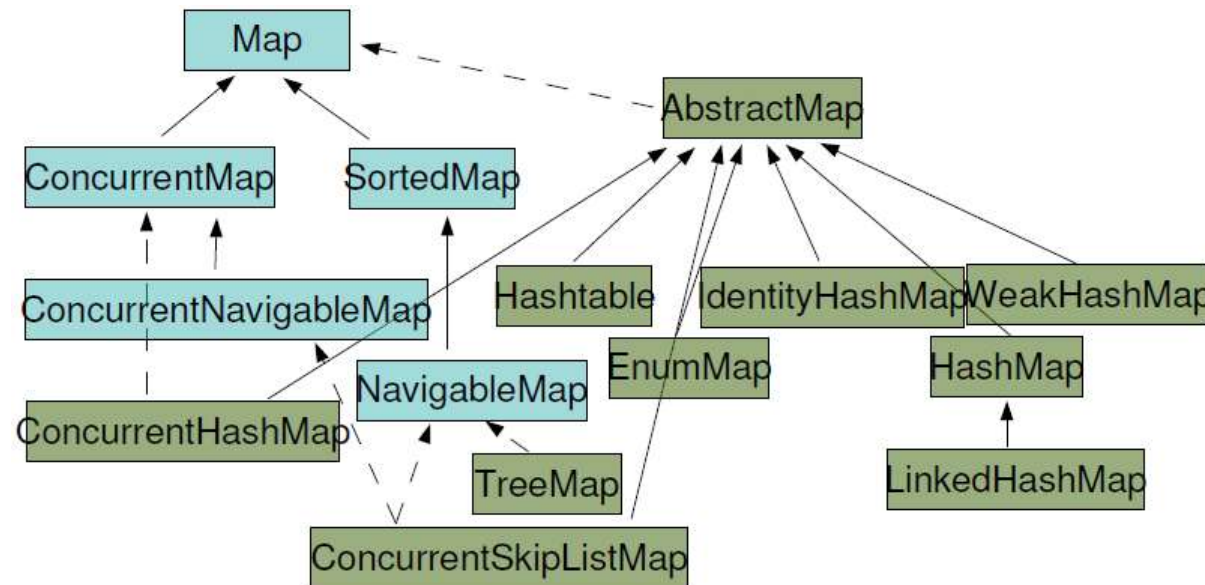
● **ArrayDeque et LinkedList fournissent les méthodes :**

| boolean add(E e)                       | E remove()                                        | E element()                                            | Lever une exception si problème                               |
|----------------------------------------|---------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------|
| boolean offer(E e)                     | E poll()                                          | peek()                                                 | Retourner une valeur spéciale (false ou null) en cas d'échec. |
| Ajouter un élément à la fin de la file | Retirer et retourner l'élément en tête de la file | Consulter l'élément en tête de la file sans le retirer |                                                               |



## 2- Collections en Java

### 2.3.2.4 Implémentations : Map



#### ● Mêmes types d'implémentations que pour « Set » :

- **HashMap** : performante mais aucune garantie en termes d'ordre
- **LinkedHashMap** : une HashMap qui préserve l'ordre d'insertion des clés dans la map.
- **TreeMap** : les clés sont triées

## 2- Collections en Java

### 2.3.3.1 Itération sur les collections (1/4)



- La boucle **foreach** est un moyen simple et pratique pour itérer sur les éléments d'une collection.

```
Collection<Type> col = new ArrayList<>();
for (Type elem : col) {
 System.out.println(elem);
}
```

- Java utilise en interne la méthode **iterator()** de la collection pour obtenir un itérateur et itérer sur les éléments de la collection.
- Depuis Java 8, la méthode **forEach()** permet d'itérer sur les éléments d'une collection.

```
List<Integer> l = new LinkedList<>();
l.forEach(e -> System.out.println(e));
```

## 2- Collections en Java

### 2.3.3.1 Itération sur les collections (2/4)



- La boucle foreach et la méthode `forEach()` **interdisent les modifications de la collection pendant l'itération**. Si on tente de modifier la structure de la collection (comme ajouter ou supprimer des éléments) pendant l'itération, une **`ConcurrentModificationException`** sera levée à l'exécution pour assurer la cohérence de l'itération.

```
List<Integer> liste = new LinkedList<>();
for (Integer element : liste) {
 liste.add(5);
 // une exception de type
 // java.util.ConcurrentModificationException
 // sera levée à l'exécution
 // car la modification de la structure de la
 // collection pendant l'itération est interdite.
}
```

- Lorsqu'on a besoin d'itérer sur une collection et potentiellement de la modifier en même temps, il est recommandé d'utiliser un itérateur explicite obtenu à partir de la méthode `iterator()`

## 2- Collections en Java

### 2.3.3.1 Itération sur les collections (3/4)



- Cela permet d'itérer sur les éléments de la collection tout en utilisant les méthodes de l'itérateur telles que `remove()` pour supprimer des éléments de la collection en toute sécurité.

```
Collection<Type> c = ... ;
for (Iterator<Type> it = c.iterator(); it.hasNext();) {
 Type o = it.next();
 if (o.getValeur() > 10) //On suppose que la méthode
 //getValeur() existe
 it.remove(); } //Supprime l'objet o de la collection c
```

- Depuis Java 8, l'interface `Collection` dispose d'une méthode **`removeIf()`** qui permet de supprimer des éléments de la collection en fonction d'un prédicat. Cela permet de supprimer des éléments de manière conditionnelle sans avoir à utiliser un itérateur explicite.

```
c.removeIf(o -> o.getValeur() > 10);
```

## 2- Collections en Java

### 2.3.3.1 Itération sur les collections (4/4)



- Contrairement aux collections, les **Maps n'implémentent pas l'interface Iterable**, ce qui signifie qu'elles ne peuvent pas être utilisées directement dans une boucle foreach.

- Pour itérer sur une Map, on peut utiliser une boucle foreach explicite avec entrySet() ou keySet() :

```
for (Map.Entry<String, Integer> i : map.entrySet())
 System.out.println(i.getKey()+":"+ i.getValue());
```

- ou utiliser la méthode forEach() depuis Java 8 avec une expression lambda pour parcourir les entrées de la Map :

```
map.forEach((k,v) -> System.out.println((k+":"+v)));
```

## 2- Collections en Java

### 2.3.3.2 Méthodes à redéfinir pour utiliser les collections



- Certaines collections comme **HashSet** et **HashMap**, utilisent des structures internes telles que des **tables de hash** pour stocker leurs éléments.
  - Pour assurer leur bon fonctionnement, il est essentiel de redéfinir les méthodes **equals()** et **hashCode()**
    - La méthode **equals(Object)** détermine si deux objets sont égaux, tandis que la méthode **hashCode()** calcule le code de hachage d'un objet, qui est utilisé comme indice dans la structure de hash interne.
      - Ces deux méthodes ont des implémentations par défaut basées sur les adresses des objets et pas leurs contenus.
    - Il est crucial que les objets égaux aient le même code de hachage et que les objets non égaux puissent avoir des codes de hachage identiques ou différents.



## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (1/6)



- Certaines collections ont un ordre à respecter : **TreeSet**, **PriorityQueue** et **TreeMap**.
  - L'ordre naturel est défini par l'interface **Comparable**
- Pour utiliser l'interface générique **Comparable<E>** :
  1. Implémenter l'interface :

```
public class MaClasse implements Comparable<MaClasse>
```
  2. Définir la méthode **int compareTo(MaClasse o)**
  3. Redéfinir les méthodes boolean **equals(Object o)** et **int hashCode()**
    - L'ordre naturel (imposé avec la méthode **compareTo()**) doit être cohérent avec la méthode **equals()**.







## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (2/6)



```
import java.util.*;
class Ville implements Comparable<Ville> {
 private String nom;
 private int code;
 public Ville(String nom, int code) {
 this.nom = nom;
 this.code = code;
 }
 public String toString() { return nom; }
 public int compareTo(Ville ville) {
 return code - ville.code;
 }
 public boolean equals(Object o) {
 if (this == o) return true;
 if (o == null || getClass() != o.getClass())
 return false;
 Ville ville = (Ville) o;
 return code == ville.code;
 }
 public int hashCode() {
 return Objects.hash(code);
 }
}
```





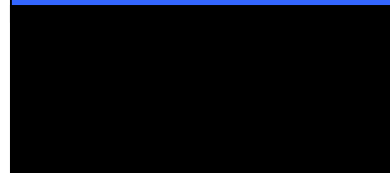
## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (3/6)



```
public class TreeVille {
 public static void main(String[] args) {
 Ville tu = new Ville("Tunis", 71);
 Ville ga = new Ville("Gabes", 75);
 Ville sf = new Ville("Sfax", 74);
 Set<Ville> s = new TreeSet<>();
 s.add(ga);
 s.add(tu);
 s.add(sf);
 s.forEach(e -> System.out.println(e));
 }
}
```

Résultat d'affichage



## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (4/6)



- **L'interface Comparator permet de trier une collection donnée de différentes façons.**
- **Pour ordonner les objets d'une collection selon différents attributs (id, nom, age, code, ...), il faudra utiliser l'interface Comparator.**
  - L'utilisation de l'interface Comparator est appropriée pour ordonner les objets d'une collection selon différents attributs lorsque la méthode compareTo() de l'interface Comparable est déjà utilisée (impossible de l'implémenter une deuxième fois) ou pour définir un ordre de tri différent de l'ordre naturel défini par la méthode compareTo().
- **L'interface Comparator permet de définir un ordre sur les objets sans que cet ordre devienne une propriété de l'objet.**





## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (5/6)



```
class Ville{
 private String nom;
 private int code;
 public Ville(String nom, int code) {
 this.nom = nom;
 this.code = code;
 }
 public String getNom() { return nom; }
 public int getCode() { return code; }
 public String toString() { return nom; }
}
```

```
import java.util.Comparator;
class OrdreSelonCode implements Comparator<Ville>{
 public int compare(Ville v1, Ville v2) {
 return v1.getCode() - v2.getCode();
 }
}
```

```
import java.util.Comparator;
class OrdreSelonNom implements Comparator<Ville>{
 public int compare(Ville v1, Ville v2) {
 return v1.getNom().compareTo(v2.getNom());
 }
}
```

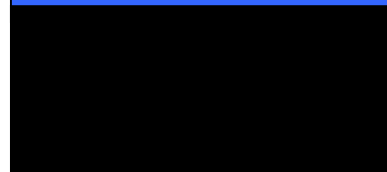
## 2- Collections en Java

### 2.3.3.3 Ordonner les objets des collections (6/6)



```
import java.util.TreeSet;
public class TestComparator {
 public static void main(String[] args) {
 Ville tu = new Ville("Tunis", 71);
 Ville ga = new Ville("Gabes", 75);
 Ville sf = new Ville("Sfax", 74);
 TreeSet<Ville> s = new TreeSet<>(new OrdreSelonNom());
 //TreeSet<Ville> s = new
 // TreeSet<>(Comparator.comparing(Ville::getNom));
 //TreeSet<Ville> s1 = new TreeSet<>(new OrdreSelonCode());
 //TreeSet<Ville> s1 = new
 // TreeSet<>(Comparator.comparingInt(Ville::getCode));
 s.add(tu);
 s.add(sf);
 s.add(ga);
 s.forEach(e -> System.out.println(e));
 }
}
```

#### Résultat d'affichage



## 2- Collections en Java

### 2.3.4.1 Algorithmes : Tri



#### ● Les tableaux ou collections peuvent être facilement triés en utilisant des méthodes simples :

- Trier un tableau tab → utiliser la méthode « **sort** » de la classe **Arrays**

- Arrays.sort(tab)

- Trier une collection → utiliser les méthodes « **sort** » de la classe **java.util.Collections**

- Ne fonctionne qu'avec les collections dérivant de l'interface **List**

- Si les éléments de la collection sont comparables selon un ordre naturel spécifique (implémentent l'interface « Comparable »)

- Collections.sort(objList)

- Si les éléments de la collection ne sont pas comparables, il faut alors spécifier quelle classe servira de comparateur

- Collections.sort(objList, objComparator)



## 2- Collections en Java

### 2.3.4.2 Algorithmes : Autres (1/2)



● D'autres opérations sont fournies par JCF et définies dans la classe **Collections** qui contient uniquement des méthodes statiques pour manipuler des collections (objets de type **Collection**) :

- Recherche binaire
  - `Collections.binarySearch(liste, clé)`
- Mélange
  - `Collections.shuffle(liste)`
- Inversion de l'ordre
  - `Collections.reverse(liste)`
- Réinitialisation des éléments (remplace tous les éléments par l'objet spécifié)
  - `Collections.fill(liste, objetParDefaut)`

## 2- Collections en Java

### 2.3.4.2 Algorithmes : Autres (2/2)



- Copie des éléments d'une liste dans une autre
  - Collections.copy(listeSource, listeDestination)
- Recherche d'extrema
  - Selon l'ordre naturel des éléments Collection.min(liste) et Collections.max(liste) ou selon un comparateur défini au préalable Collections.min(liste, comparateur) et Collections.max(liste, comparateur)
- **La classe Collections fournit également des méthodes statiques pour manipuler des collections non modifiables telles que :**
  - unmodifiableList(List<? extends T> list): Renvoie une vue non modifiable de la liste spécifiée.
  - unmodifiableSet(Set<? extends T> s): Renvoie une vue non modifiable du Set spécifié.
  - unmodifiableMap(Map<? extends K, ? extends V> m)  
Renvoie une vue non modifiable de la map spécifiée.



## 2- Collections en Java

### 2.3.5 Collections immuables (1/2)



- Toutes les classes du JCF (ArrayList, LinkedList, HashSet, TreeMap, etc.) sont supposées modifiables. Pour obtenir une collection immuable (ou non modifiable), on peut soit :
  - Utiliser une **méthode** spéciale retournant une collection immuable (voir la diapo suivante).
  - Obtenir une **vue immuable** sur une collection via :
    - La classe **Collection** offre de nombreuses méthodes statiques **unmodifiableXXX()** (où XXX représente une interface d'une collection telle que Collection, List, Set y compris l'interface Map).
    - La classe **Arrays** offre la méthode statique **asList()** permettant d'obtenir une vue sur un tableau sous la forme d'une liste.
      - `static <E> List<E> asList(E... a)` : Retourne une liste partiellement immuable (possibilité d'utiliser la méthode `set`) contenant les éléments donnés. Toute utilisation d'une méthode changeant la taille de la liste (ex. `add` ou `remove`) déclenche l'exception `UnsupportedOperationException`.

## 2- Collections en Java

### 2.3.5 Collections immuables (2/2)



● **Java 9 a introduit plusieurs méthodes statiques de fabriques dans les interfaces List, Set et Map, ce qui permet de créer des objets de collections immuables.**

● **List.of(), Set.of(), Map.of() et Map.ofEntries()** sont des méthodes statiques qui offrent un moyen pratique pour créer des collections immuables.

- static <E> List<E> of(E... elements)
- static <E> Set<E> of(E... elements)
- static <K,V> Map<K,V> of(K k1, V v1, K k2, V v2, ...) : Crée un objet Map immuable à partir de paires clé et valeur.
- static <K,V> Map<K,V> ofEntries(Map.Entry<? extends K, ? extends V>... entries) : Crée un objet Map immuable à partir des objets Map.Entry ou entrées spécifiés.
- static <K,V> Map<K,V> copyOf(Map<? extends K,? extends V> m) : Crée un objet Map immuable à partir d'un objet Map spécifié. Cette méthode est proposée depuis Java 10.

## 2- Collections en Java

### 2.3.6 Collections séquencées (1/6)



- Le JCF manque d'un type de collection qui représente une **séquence d'éléments avec un ordre de rencontre défini**. L'objectif est d'introduire de nouvelles **interfaces** pour représenter les collections avec un ordre de rencontre défini et définir un ensemble uniforme d'opérations qui s'appliquent à ces collections.

- Chacune de ces collections possède un premier élément bien défini, un deuxième élément, et ainsi de suite jusqu'au dernier élément. Elle fournit aussi des méthodes pour accéder à son premier et dernier élément, et pour traiter ses éléments dans l'ordre inverse.
- **List** et **Deque** définissent tous deux un ordre de rencontre, mais leur supertype **Collection**, ne le définit pas. De même, **Set** ne définit pas d'ordre de rencontre et les sous-types tels que **HashSet** n'en définissent pas, alors que les sous-types **SortedSet** et **LinkedHashSet** en définissent un.



## 2- Collections en Java

### 2.3.6 Collections séquencées (2/6)



- Une **collection séquencée** est une collection dont les éléments ont un ordre de rencontre défini.

```
interface SequencedCollection<E> extends Collection<E> {
 void addFirst(E);
 void addLast(E);
 E getFirst();
 E getLast();
 E removeFirst();
 E removeLast();
 SequencedCollection<E> reversed(); }

```

- Les six premières méthodes sont issues de Deque. Elles permettent d'ajouter, d'obtenir et de supprimer des éléments aux deux extrémités.
- La nouvelle méthode **reversed()** permet d'obtenir **une vue** inversée de la collection d'origine. Toute modification apportée à la collection d'origine est visible dans la vue, et vice versa si cela est autorisé.



## 2- Collections en Java

### 2.3.6 Collections séquencées (3/6)



- Un **ensemble séquencé** est un ensemble ne contenant pas d'éléments dupliqués.

interface **SequencedSet<E>** extends Set<E>, SequencedCollection<E> {  
SequencedSet<E> reversed(); }

- Les collections telles que **SortedSet** ne prennent pas en charge les opérations de positionnement explicite comme l'ajout en tête ou en queue, car elles maintiennent un ordre spécifique basé sur le tri des éléments. Par conséquent, les méthodes **addFirst(E)** et **addLast(E)**, déclarées dans la super interface SequencedCollection, **ne sont pas fonctionnelles** dans le contexte des collections comme SortedSet.
- Les méthodes **addFirst(E)** et **addLast(E)** de SequencedSet ont une sémantique particulière pour les collections telles que **LinkedHashSet**.
  - Si l'élément est déjà présent dans l'ensemble, il est déplacé à la position appropriée. Cela permet de remédier à une lacune de LinkedHashSet classique, à savoir l'impossibilité de repositionner des éléments.



## 2- Collections en Java

### 2.3.6 Collections séquencées (4/6)



- Une **Map séquencée** est une Map dont les entrées ont un ordre de rencontre défini.

```
interface SequencedMap<K,V> extends Map<K,V> {
 Entry<K, V> firstEntry();
 Entry<K, V> lastEntry();
 Entry<K, V> pollFirstEntry();
 Entry<K, V> pollLastEntry();
 SequencedMap<K,V> reversed();
 SequencedSet<K> sequencedKeySet();
 SequencedCollection<V> sequencedValues();
 SequencedSet<Entry<K,V>> sequencedEntrySet();
 V putFirst(K, V);
 V putLast(K, V);
}
```

- Les quatre premières méthodes de SequencedMap sont issues de **NavigableMap**. Elles permettent d'obtenir et de supprimer des entrées aux deux extrémités.

## 2- Collections en Java

### 2.3.6 Collections séquencées (5/6)



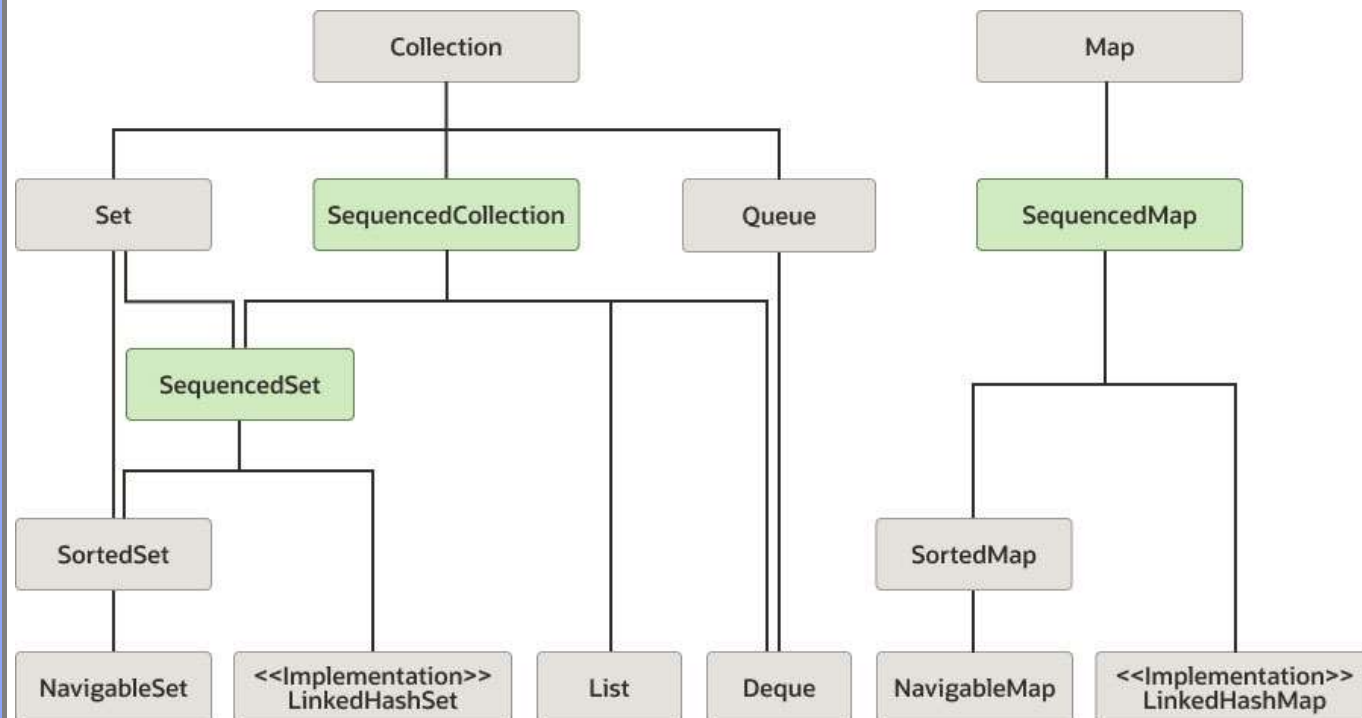
- Les méthodes `putFirst()` et `putLast()` ont une sémantique particulière, similaire aux méthodes `addFirst()` et `addLast()` de `SequencedSet`.
  - Pour **LinkedHashMap**, ces méthodes ont pour effet supplémentaire de repositionner l'entrée si elle est déjà présente dans la Map.
  - Pour **SortedMap**, ces méthodes provoquent une exception de type `UnsupportedOperationException`.
- Trois méthodes sont ajoutées à la classe **Collections** afin de créer des collections séquencées immuables :
  - `Collections.unmodifiableSequencedCollection(sequencedCollection)`
  - `Collections.unmodifiableSequencedSet(sequencedSet)`
  - `Collections.unmodifiableSequencedMap(sequencedMap)`





## 2- Collections en Java

### 2.3.6 Collections séquencées (6/6)



Extrait du nouveau JCF depuis Java 21





## 2- Collections en Java

### 2.4 Stream



- **L'API Stream est introduite depuis Java 8. Elle est utilisée pour traiter des collections d'objets.**
  - Un stream (flux) est une séquence d'objets qui prend en charge diverses méthodes qui peuvent être enchaînées pour produire le résultat souhaité.
- **Les fonctionnalités du Stream en Java sont les suivantes :**
  - Stream prend une collection (pas de Map), un tableau ou des E/S en invoquant la méthode :  
default Stream<E> **stream()**
  - Les Stream ne modifient pas la structure de données d'origine. Ils fournissent uniquement le résultat selon les méthodes enchaînées.
  - Chaque **opération intermédiaire** est exécutée et renvoie un stream en conséquence, de sorte que diverses opérations intermédiaires peuvent être enchaînées. Les **opérations terminales** marquent la fin du stream et renvoient le résultat.

## 2- Collections en Java

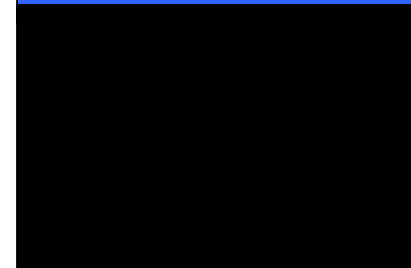
### 2.4 Stream : Exemple 1



```
import java.util.*;

public class Exemple1{
 public static void main(String[] args){
 List<Integer> list = Arrays.asList(1, 2, 3, 4);
 list.stream()
 .map(x -> x + 1)
 .forEach(System.out::println);
 }
}
```

Résultat d'affichage



- La méthode **map()** (opération intermédiaire) est utilisée pour renvoyer un flux constitué en appliquant une fonction fournie en paramètre pour transformer l'élément de départ en créant un nouveau.
- La méthode **forEach()** (opération terminale) est utilisée pour parcourir chaque élément du flux.



## 2- Collections en Java

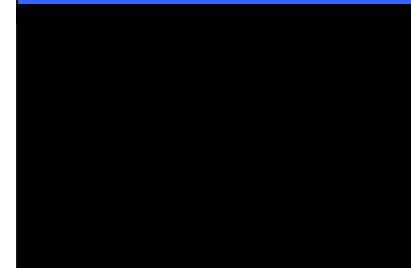
### 2.4 Stream : Exemple 2



```
import java.util.*;

public class Exemple2{
 public static void main(String[] args){
 List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
 list.stream()
 .filter(num -> num % 2 == 0)
 .forEach(System.out::println);
 }
}
```

Résultat d'affichage



- La méthode **filter()** (opération intermédiaire) est utilisée pour sélectionner les éléments selon le prédicat passé en argument.



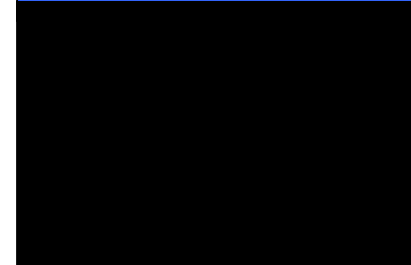
## 2- Collections en Java

### 2.4 Stream : Exemple 3



```
import java.util.*;
public class Exemple3{
 public static void main(String[] args){
 List<Integer> list = Arrays.asList(6, 2, 9, 7);
 list.stream()
 .sorted()
 .forEach(System.out::println);
 }
}
```

Résultat d'affichage



- La méthode **sorted()** (opération intermédiaire) est utilisée pour trier un flux.



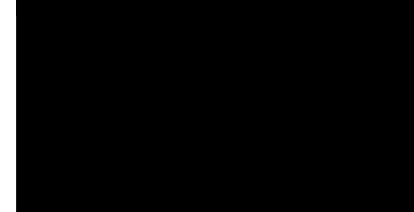
## 2- Collections en Java

### 2.4 Stream : Exemple 4

```
import java.util.*;
import java.util.stream.Collectors;

public class Exemple4{
 public static void main(String[] args){
 List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
 List<Integer> pair = list.stream()
 .filter(x -> x % 2 == 0)
 .collect(Collectors.toList());
 pair.forEach(System.out::println);
 }
}
```

Résultat d'affichage



- La méthode **collect()** (opération terminale) est utilisée pour renvoyer le résultat des opérations intermédiaires effectuées sur le flux.
- Elle est utilisée avec un objet Collector fourni par la méthode `Collectors.toList()`, qui collecte les éléments du flux dans une liste. L'interface Collector est utilisée pour définir une opération de collecte lors du traitement d'un Stream.



## 2- Collections en Java

### 2.4 Stream : Exemple 5



```
import java.util.*;
import java.util.stream.Collectors;
record Humain(String nom, int age, char sexe) { }
public class TestHumain {
 public static void main(String[] args) {
 List<Humain> liste = Arrays.asList(
 new Humain("Bechir", 27, 'M'),
 new Humain("Bassma", 25, 'F'),
 new Humain("Mourad", 24, 'M')
);
 Map<Character, List<Humain>> humainsParSexe = liste.stream()
 .collect(Collectors.groupingBy(Humain::sexe));
 System.out.println(humainsParSexe);
 }
}
```

#### Résultat d'affichage

la méthode **groupingBy()** fournie par Collectors pour regrouper les humains par sexe.



## 2- Collections en Java

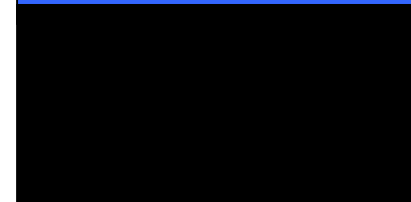
### 2.4 Stream : Exemple 6



```
import java.util.*;

public class Exemple5{
 public static void main(String[] args){
 int[] nbr = {1, 2, 3};
 int somme = Arrays.stream(nbr)
 .reduce(0, (a, b) -> a + b);
 System.out.println(somme);
 }
}
```

Résultat d'affichage



- La méthode **reduce()** (opération terminale) est utilisée pour combiner les éléments d'un flux en une seule valeur en appliquant une opération de réduction. Elle prend un **BinaryOperator** en paramètre, qui est une interface fonctionnelle représentant une opération binaire prenant deux opérandes de même type et renvoyant un résultat de ce type.

● Pour aller plus loin, consulter l'API Stream.

